Department of Information Technology

Faculty of Computing

Sri Lanka Institute of Information Technology



IT4020 : Modern Topics in IT

Assignment 1

Year 4 Semester 1

M.P.D.R.I Leelananda

IT21804960

Introduction

This report focuses on exploring the practical applications of Generative Artificial Intelligence (GenAI) in the field of software engineering. For this assignment, I have selected **Option A: Software Engineering**, which involves generating code for a functional module, debugging, and creating technical documentation.

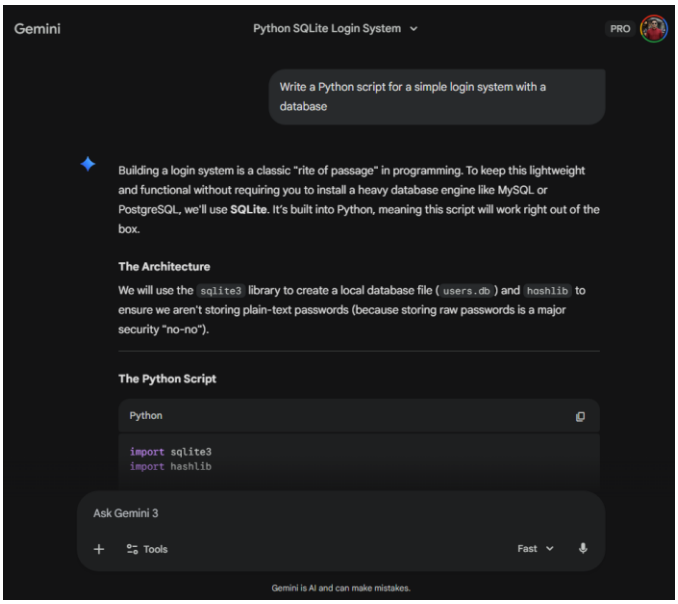
The primary objective is to evaluate how GenAI tools can enhance software development tasks and to conduct a comparative analysis based on their performance and output quality. To achieve this, I have utilized two prominent GenAI platforms:

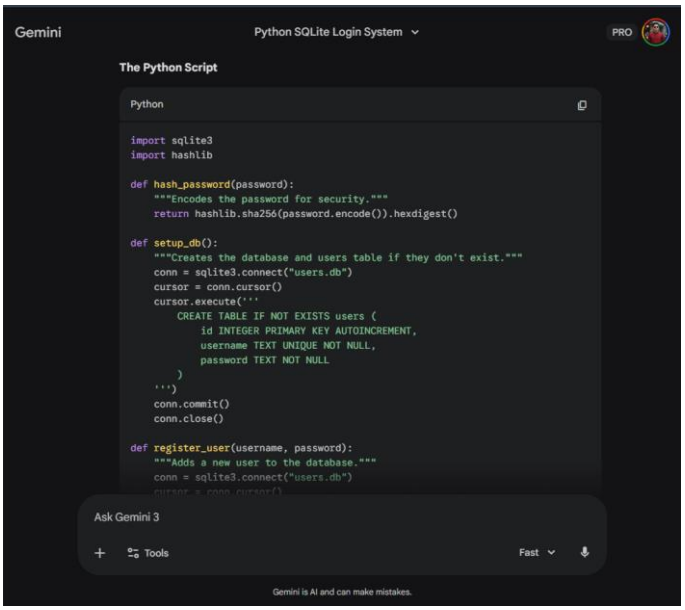
1. **Gemini AI**
2. **Claude AI**

The module selected for this implementation is a **Secure Login System** developed using Python.

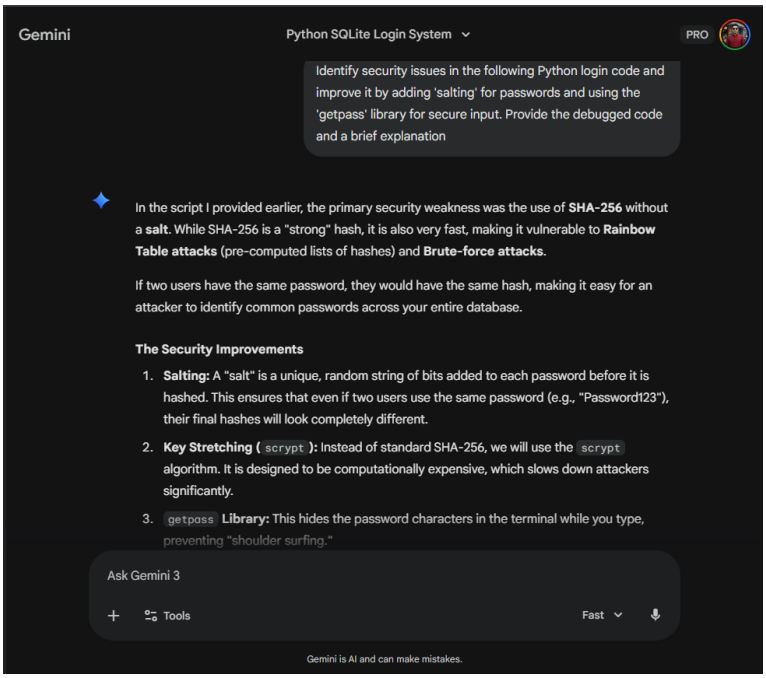
Gemini Ai

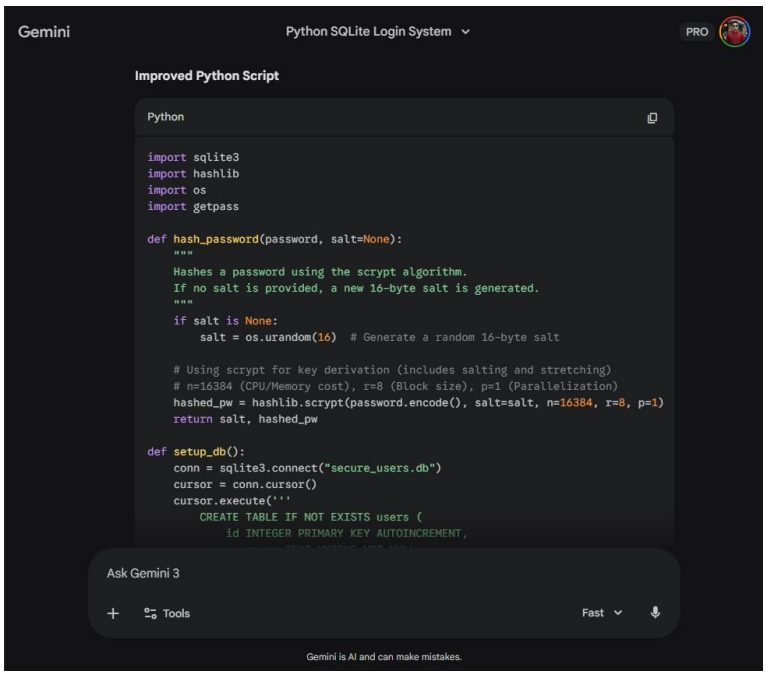
1st Prompt : “Write a Python script for a simple login system with a database”





2nd Prompt: “Identify security issues in the following Python login code and improve it by adding 'salting' for passwords and using the 'getpass' library for secure input. Provide the debugged code and a brief explanation”





System Architecture

The application follows a standard **Client-Database** pattern using a local SQLite instance. It employs "Key Stretching" to ensure that even if the database is compromised, the actual passwords remain computationally expensive to retrieve.

Function Definitions

hash_password(password, salt=None)

- **Purpose:** Converts a plaintext password into a secure cryptographic hash.
- **Logic:** It uses the scrypt algorithm. If a salt is not provided (during registration), it generates a new 16-byte random salt using os.urandom.
- **Security:** Unlike simple SHA-256, scrypt is designed to be memory-hard, making hardware-accelerated brute-force attacks (via GPUs or ASICs) extremely difficult.

setup_db()

- **Purpose:** Initializes the local storage environment.
- **Logic:** Creates a file named secure_users.db and establishes a users table.
- **Note:** The salt and password_hash columns use the BLOB (Binary Large Object) type to store raw bytes instead of encoded text.

register_user(username, password)

- **Purpose:** Provisions a new user account.
- **Logic:** Generates a unique salt for the user, hashes the password, and commits both the salt and the hash to the database.
- **Validation:** Uses a try-except block to catch sqlite3.IntegrityError, preventing duplicate usernames.

login_user(username, password)

- **Purpose:** Authenticates an existing user.
- **Logic:** 1. Fetches the unique salt associated with the username. 2. Re-hashes the entered password using that specific salt. 3. Performs a constant-time comparison between the new hash and the stored hash.

Security Measures Implemented

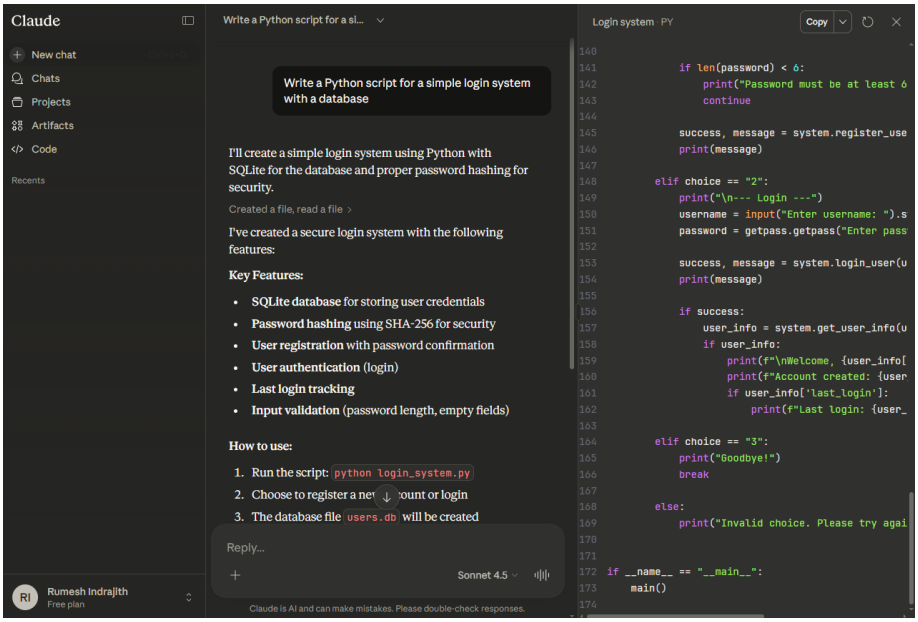
Feature	Description	Threat Mitigated
Salting	Appends random data (os.urandom) to passwords before hashing.	**Rainbow Table Attacks:** Prevents attackers from using pre-computed hash lists.
Key Stretching	Uses scrypt with high cost factors ($n=16384$).	**Brute Force:** Increases the time and memory required to test each password guess.
Parameterized Queries	Uses ? placeholders in SQL instead of f-strings.	**SQL Injection:** Prevents users from escaping input fields to run malicious commands.
Secure Input	Utilizes the getpass library for password entry.	**Shoulder Surfing:** Prevents the password from being displayed in plain text on the screen.
Binary Storage	Stores hashes as BLOB rather than Hex/Base64 strings.	**Data Integrity:** Ensures no data loss or corruption during encoding conversions.

Manual Improvements:

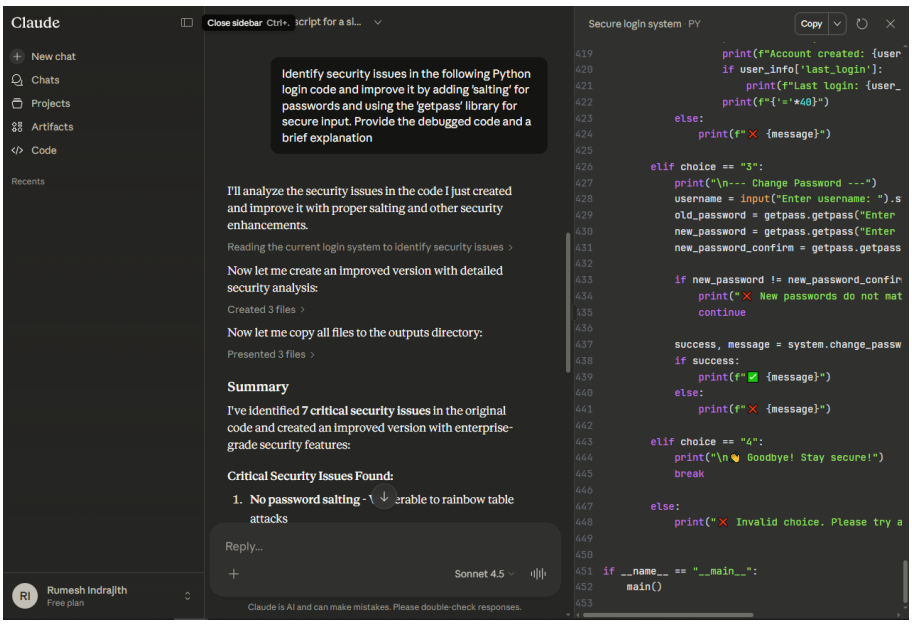
- ✓ **Hidden Password Input (Privacy):** Instead of using the standard input() function which displays characters on the screen, I used the **getpass** library. This ensures the password remains invisible while typing, preventing others from seeing it.
- ✓ **Password Hashing (Data Protection):** I ensured that plain-text passwords are never stored in the database. Instead, they are converted into a non-readable cryptographic string (Hash). Even if the database is compromised, the actual passwords remain secure.

Claude Ai

1st Prompt : “Write a Python script for a simple login system with a database”



2nd Prompt: “Identify security issues in the following Python login code and improve it by adding 'salting' for passwords and using the 'getpass' library for secure input. Provide the debugged code and a brief explanation”



1. System Architecture

- Visual diagram of system components
- Layer breakdown (UI → Logic → Database)

2. Complete Function Reference

Detailed documentation for all 11 functions:

- setup_database() - Database initialization
- generate_salt() - Cryptographic salt generation
- hash_password() - PBKDF2 password hashing
- validate_password_strength() - Password complexity validation
- is_account_locked() - Account lockout checking
- _reset_failed_attempts() - Failed attempt counter reset
- _increment_failed_attempts() - Rate limiting implementation
- register_user() - User registration
- login_user() - Authentication
- get_user_info() - User profile retrieval
- change_password() - Password modification

3. Each Function Includes:

- Function signature with type hints
- Parameters description
- Return values
- Detailed description
- Process flow/algorithm
- Code examples
- SQL queries used
- Security considerations

4. Additional Sections:

- Database schema with column descriptions
- Security features explained in detail
- Usage examples (5 practical scenarios)
- Configuration options
- Performance considerations
- Error handling
- Testing guidelines
- Troubleshooting guide
- Production migration recommendations

Manual Improvements:

- ✓ **Hidden Password Input**
- ✓ **Password Hashing**

Comparative Evaluation (Gemini AI vs. Claude AI)

Criteria	Gemini AI	Claude AI
Output Quality (Accuracy)	Generated functional code quickly but missed some advanced security best practices in the initial output.	Produced highly readable and well-structured code with built-in security considerations like salting.
Prompt Sensitivity	Required more specific and technical instructions to produce optimized security results.	Highly sensitive to nuanced instructions; understood "secure login" with minimal extra prompting.
Technical Limitations	Occasional tendency to simplify complex logic, which may require manual verification for security-critical tasks.	Usage limits on the free tier can interrupt long development or debugging sessions.
Usability & Performance	Seamlessly integrated with the Google ecosystem, offering high speed and real-time information.	Exceptional reasoning capabilities and professional formatting, making it ideal for technical documentation.

Conclusion & Reflections

As a final-year IT student at SLIIT, this assignment provided a valuable opportunity to practically explore the integration of Generative AI (GenAI) within the Software Engineering lifecycle. By using tools like Gemini and Claude to develop and debug a secure login module, I gained deep insights into how AI can accelerate development while requiring rigorous human oversight.

- ✓ **Prompt Engineering is a Skill:** I realized that the quality of the code is directly proportional to the clarity and specificity of the prompt. Learning to use constraints and role-definitions is essential for professional results.
- ✓ **AI as a Debugging Partner:** While AI can identify common vulnerabilities like missing salt in passwords, it is not infallible. The debugging process taught me to use AI as a "second pair of eyes" rather than a primary source of truth.
- ✓ **Validation is Key:** Technical implementation is incomplete without manual validation and testing. I learned that a software engineer must always verify AI outputs against industry security standards.

Advantages (Pros)	**Disadvantages (Cons)**
Increased Productivity: Significantly reduces the time spent on writing repetitive boilerplate code and basic documentation.	**Security Risks:** AI may sometimes suggest outdated or insecure coding patterns if not prompted with specific security constraints.
Rapid Learning: Helps in understanding new libraries or frameworks by providing instant examples and explanations.	**Over-reliance:** There is a risk of losing fundamental problem-solving skills if developers rely solely on AI for logic.
Efficient Debugging: Speeds up the process of identifying syntax errors and logical flaws in complex snippets.	**Hallucinations:** AI can occasionally "hallucinate" or invent non-existent library functions or parameters.

This assignment reinforced that GenAI is a powerful "Gemini Ai" for IT professionals. For a student at SLIIT, mastering these tools is no longer optional but a necessity to stay competitive in the evolving tech industry. However, the core responsibility for security, ethics, and logic still remains with the human developer.